

Low Cost FPGA-based Bit Error Rate Test set

Vishnu Barath

3rd Year Project Interim Report

Department of Electronic &
Electrical Engineering

UCL

Supervisor: Dr Martyn Fice

December 8, 2025

I have read and understood UCL's and the Department's statements and guidelines concerning plagiarism.

I declare that all material described in this report is my own work except where explicitly and individually indicated in the text. This includes ideas described in the text, figures and computer programs.

This report contains _ pages (excluding this page and the appendices) and _ words.

Signed: Vishnu Barath

Date: December 8, 2025

Abstract

The main aim of this project is to build a low-cost system that can test the Bit Error Ratio (BER) of a communication channel. It will use Pseudorandom Binary Sequence (PRBS) generators to generate a known bitstream which would then be transmitted across a communication channel. The sequence will be generated on both the transmitter and the receiver to compare them and calculate the BER. So far, a transmitter has been successfully developed which generates a repeating PRBS-7 sequence with a length of 127 bits. The PRBS generator is implemented using a Linear Feedback Shift Register controlled by a 10MHz clock signal where a bit is transferred at every rising edge, achieving a data rate of 10Mbps. This has been implemented on an FPGA.

Contents

1	Project Description	3
2	Literature Review	4
2.1	Serial Communication BER Tester	4
2.2	VLC BER Tester	4
2.3	Microcontroller-based BER	4
2.4	Summary	5
3	Work Performed to-date	6
3.1	Choice of Hardware	6
3.1.1	MC	6
3.1.2	FPGA	8
3.1.3	Final Choice	10
3.2	PRBS generator	10
3.2.1	Push Button Clock	11
3.2.2	500kHz Clock	11
3.2.3	10MHz Clock	13
3.2.4	20MHz Clock	15
4	Project Management	17
5	Future Work	17
A	Appendices	19
A.1	Quartus Prime Lite Block Diagrams	19

1 Project Description

Communication networks are continuously expanding in capacity and speed and being able to ensure signal integrity is paramount. The most common example of such a system is the internet, which uses undersea optical fibres for data transmission. Such networks have become a central component in the modern day world with applications in many important industries, such as telecommunications, cloud services, banking, healthcare and transportation. In accordance with these advancements, techniques to measure the performance of communication channels have evolved. Channels for digital communication are susceptible to incorrect data transmission arising due to many possible factors such as noise or interference. A metric of performance for digital communication channels is the Bit Error Ratio (BER) which can be characterised via a Bit Error Rate Test (BERT). [1]

The BER of a communication channel is the proportion of bits in a data transmission which have an error. When received data has bit errors it requires either retransmission and/or extra processing from error correction algorithms. Hence, a high BER would have significant impacts on the channel's data rate capability. [1] Being able to measure this is important in ensuring a channel can transmit data quickly and efficiently. Unfortunately, systems that can measure the BER can be expensive and would be designed for data rates in the Gb/s range. As a result acquiring such equipment could prove difficult and would be excessive since such high data rates aren't always required. [2]

This project aims to construct a system that can accurately measure the BER of a Visible Light Communication (VLC) channel at data rates up to 10Mb/s, while keeping costs relatively low. Below are the main goals and objectives that need to be achieved by the end of this project.

1. Make a Pseudo Random Binary Sequence (PRBS) generator
 - To test the capability of the hardware choice
 - To set a fixed word length in bits
 - To generate a known sequence of bits that appear random
 - To reach a bitrate of up to 10Mbps
2. Develop an Error detection algorithm
 - To receive bitstream at up to 10Mb/s
 - To generate the PRBS on receiver side
 - To synchronise the PRBS with the received bit stream
 - To calculate an error rate

2 Literature Review

Building a system to measure a channel's BER can be done via any programmable device such as Microcontrollers or Field Programmable Gate Arrays (FPGA). A sequence of known bits is required for testing a BER. One common method for generating one is the Pseudorandom Binary Sequence generator (PRBS). Such sequences are generated using a Linear Feedback Shift Register (LFSR) where two specific bits in the register are fed into an XOR or XNOR gate and fed back into the Least Significant Bit (LSB) of the register. [3, 4] Low cost BERTs have been developed using such methods. [2, 5, 6] Examples of these are discussed below.

2.1 Serial Communication BER Tester

An FPGA-based device was developed to test the BER of a serial communication channel. It uses a 31-bit LFSR to generate a PRBS which would have a length of $2^{31} - 1$ bits ($\sim 2.14 \times 10^9$). [2, 3]

The system works by transmitting a PRBS to an external channel and receiving the bitstream on the same FPGA. The bits are generated, received and compared simultaneously. It is capable of testing data rates of up to 50Mbps and used the Anritsu MP8302A BERT for reference. [2] By transmitting and receiving on the same FPGA, having to synchronise bitstreams was not an issue. However, this is not good for simulating realistic transmission between two different devices.

2.2 VLC BER Tester

A PCB with logic circuits was designed to test a VLC channel. This is the same type of communication channel this project is meant for. In this case, the bit generator is simply a square wave rather than a PRBS. Effectively, the bit sequence being used to test the BERT is a sequence of alternating ones and zeros. [5]

Two separate PCBs are used for transmitting and receiving the bitstreams. This BERT can test for data rates up to 370kbps, which is significantly lower than this project's targetted data rates. As a result of using a periodic square wave, the issue of synchronising the transmitted and received bitstreams is simplified. This system has significantly smaller cost and complexity in comparison to FPGA equivalents. Unfortunately, in reality, bitstreams won't be sequential ones and zeros and would appear more random, meaning the displayed error ratio may not necessarily reflect real world conditions for data transmission. [5]

2.3 Microcontroller-based BER

An STM32 Microcontroller combined with a Complex Programmable Logic Device (CPLD) was used to make a BER tester. A specific type of communication channel is not specified. This system uses pseudorandom patterns to test a channel. [6]

A single STM32 is used for both transmitting and receiving the bitstreams and tests the error ratio at 2.048Mbps. [6] This data rate is similar to the data rate this project is targetting. Due to using only one device, this may not accurately simulate real world conditions.

2.4 Summary

Overall, both FPGAs and Microcontrollers seem as valid options in building a BER tester since they can both be programmed to transmit and receive a PRBS and compare bitstreams. In order to best simulate real world data transmission, two separate devices for transmitting and receiving would be ideal. While this may increase cost, relative to the price of BER equipment, it shouldn't be significant. [2, 5] A PRBS generator will be used to generate a bitstream that will be used for calculating the BER. Figure 1 gives an overview of the proposed system.

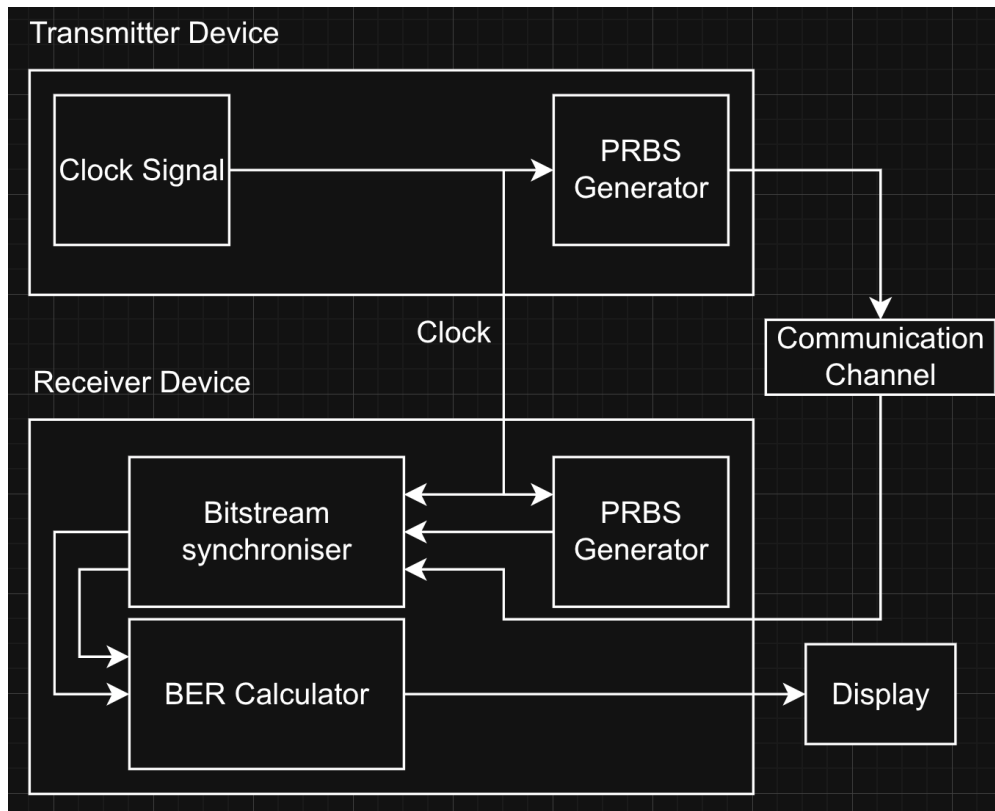


Figure 1: Block Diagram of proposed BER Testing system

3 Work Performed to-date

3.1 Choice of Hardware

In this project, there is choice of hardware between using Microcontrollers or FPGAs. To determine which is suitable, the capabilities of a device of each were tested using a 10MHz clock signal. When a bitstream is synchronised to such a clock signal, each bit would be transferred on the rising edge of the clock. This would allow for data rate of 10Mbps.

3.1.1 MC

The board being tested here is the NUCLEO-F401RE board which has an STM32F401RET6 Microcontroller using the Serial Peripheral Interface (SPI) synchronous serial communication protocol. [7, 8] It was programmed using STM32CUBEIDE.

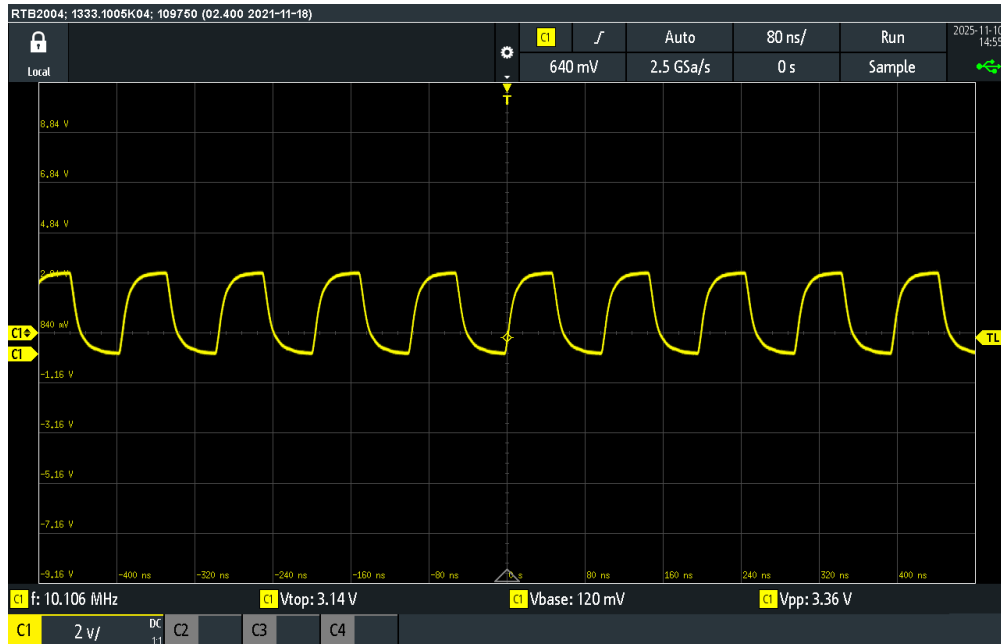


Figure 2: Oscilloscope capture of a 10MHz clock from the NUCLEO-F401RE

In Figure 2, the clock signal is the correct frequency with an adequate peak-to-peak voltage. However, the signal appears to rise and fall exponentially rather than instantaneously like a typical square wave would. Different frequencies of clock signals can be used to test for this effect.

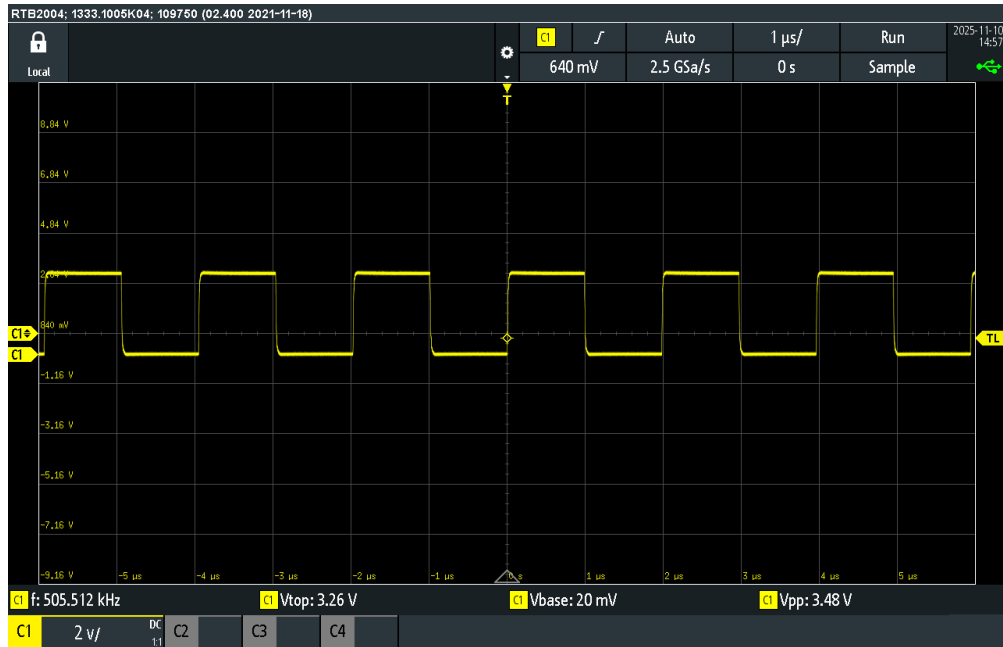


Figure 3: Oscilloscope capture of a 500kHz clock from the NUCLEO-F401RE

In Figure 3, it can be seen that the microcontroller produces a conventional square wave clock signal at 500kHz, suggesting that there is a frequency limitation in the measurement equipment.

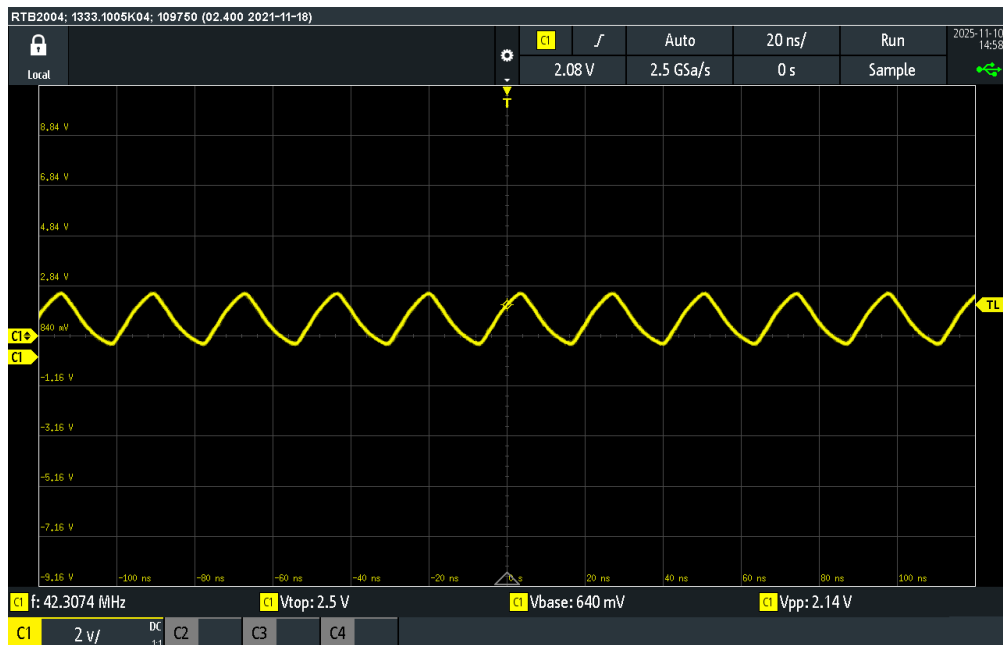


Figure 4: Oscilloscope capture of a 42MHz clock from the NUCLEO-F401RE

In Figure 4, the microcontroller is set to output a clock signal at its highest possible frequency of 42MHz. [7, 8] The signal is almost triangular, but still holds a fundamental

frequency of 42MHz. However, the peak-to-peak voltage (2.14V) is attenuated compared to that at 10MHz (3.36V) in Figure 2 and 500kHz (3.48V) in Figure 3.

The oscilloscope probe being used (RT-ZPO3) has a -3dB bandwidth of 10MHz which explains the distortion in the 10MHz and 42MHz clock signals. [9] This is because in the Fourier Series of a square wave, along with its fundamental frequency, there are other higher frequencies that allow the wave to be square. [10] So even in a 10MHz square wave, which is exactly at the cut-off frequency, there are high frequency harmonics that exist which allow for the wave to be square. These frequencies are attenuated, giving the output in Figure 2. But since the 10MHz frequency itself is within the bandwidth, there is little to no attenuation in the peak-to-peak voltage, like there is at 42MHz in Figure 4. From testing at a lower frequency, it can be concluded that the hardware still produces a square wave at 10MHz with measurements being limited by the probe.

3.1.2 FPGA

The board being tested here is a DE0-NANO which contains an ALTERA Cyclone IV EP4CE22F17C6 FPGA. It was programmed via Quartus Prime Lite. [11]

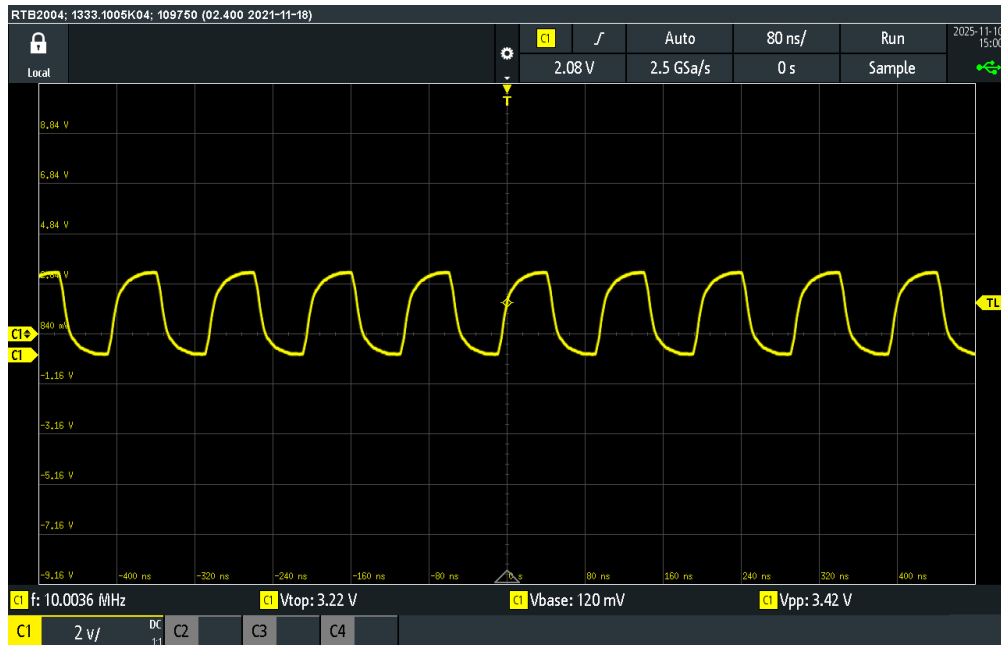


Figure 5: Oscilloscope capture of a 10MHz clock from the DE0-Nano FPGA board

Figure 5 shows that at a 10MHz clock, the FPGA provides a similar output to the microcontroller, with the probe causing the same type of distortion. There is a minor difference in peak-to-peak voltage.

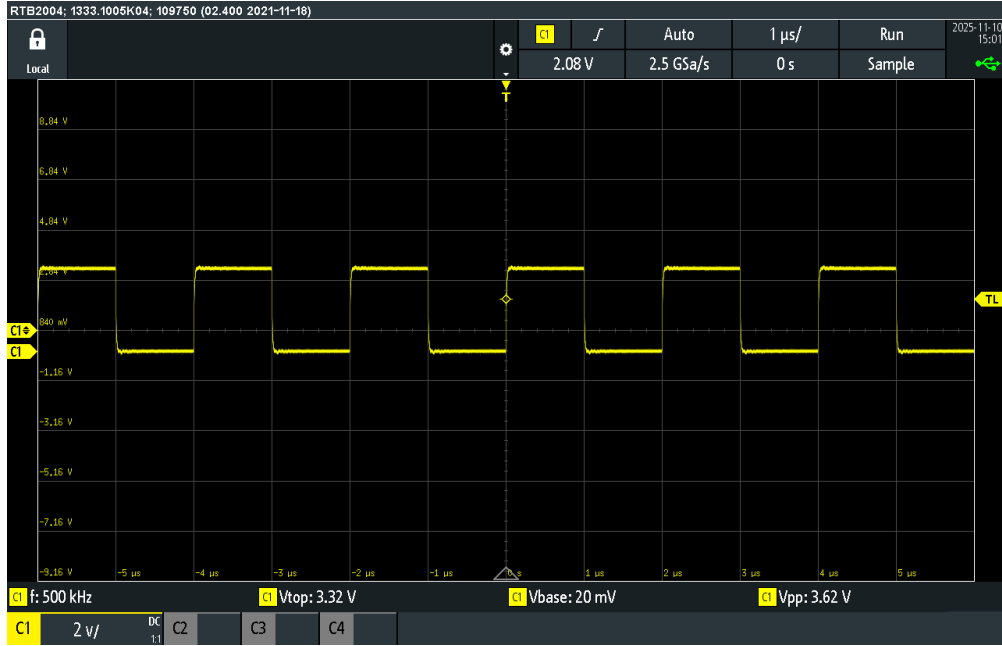


Figure 6: Oscilloscope capture of a 500kHz clock from the DE0-Nano FPGA board

When the clock is turned down to 500kHz, in Figure 6, like the microcontroller, the wave appears as a conventional square wave due to the clock frequency being well within the bandwidth of the oscilloscope probe. [9]

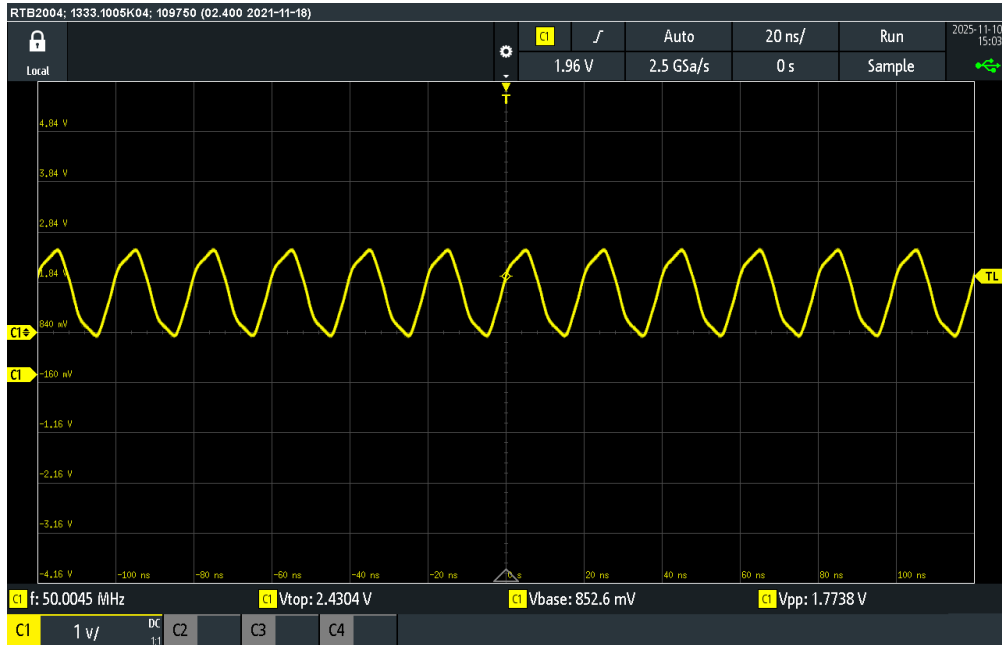


Figure 7: Oscilloscope capture of a 50MHz clock from the DE0-Nano FPGA board

The FPGA allows for a maximum clock frequency of 50MHz which is the same frequency as the onboard oscillator. [11] The clock signal in Figure 7 faces similar issues to the 42MHz

clock from the microcontroller (distortion and attenuation) but a slightly greater degree due to being a higher frequency. Since none of these effects occur at a lower frequency, this appears to be a limitation of the probe being used, not the board.

3.1.3 Final Choice

Both choices of hardware display similar results in testing with clock signals suggesting they are both adequately capable of being used to develop a BERT set. The FPGA has a higher maximum clock frequency of 50MHz over the microcontroller's 42MHz suggesting it can be pushed further to test higher data rates. [7, 8, 11]

Microcontrollers tend to be programmed in higher level programming languages than FPGAs, meaning they are easier to work with. However, this provides minimal control over the lower level logic. FPGAs allow for a much higher degree of control by working directly at the logic circuit level, but this makes them harder to program than microcontrollers.

Microcontrollers also have many peripherals you can work with, including UART, ADCs and serial communication interfaces such as I2C and SPI. These peripherals are designed for transferring bitstreams synchronised to a clock signal. On the other hand, these peripherals have complex workings which means a certain clock rate may not directly translate to the same data rate. Transferring one bit may take more than one clock cycle. FPGAs don't have any of these peripherals since they work directly at the logic circuit level. Despite being difficult to program, logic circuits can be designed in a way that allows for one bit to be transferred per clock cycle. [8, 11] When working with registers, a low level of programming is required for direct control. Microcontrollers tend to be much cheaper than FPGAs, and achieving low costs is one of the aims of this project. However, in comparison to the price of industry standard BER equipment, this difference in price is small, meaning despite the choice of hardware here, the costs will most likely be relatively low. [2, 5]

In this project, a Linear Shift Feedback Register would be required for PRBS generation, meaning direct control over logic is necessary. The clock rates available are in the Megahertz range rather than Gigahertz, so it would be ideal to make efficient use of clock cycles. Considering these factors, the FPGA has been chosen to develop the BERT.

3.2 PRBS generator

For this project, a PRBS generator will be used to generate a bitstream for the BERT. At this point in time, a PRBS-7 generator has been made. A PRBS-7 would repeat every 127 bits and is represented by the following polynomial. [3]

$$x^7 + x^6 + 1 \tag{1}$$

This polynomial isn't mathematical, it is a representation of the LFSR that would be used to generate a PRBS-7. The values of the powers represent which bits are used for the feedback. What this polynomial means is that the 7th and 6th bits of the register are inputs to an XOR gate where the output is fed into the Least Significant Bit (LSB) of the shift register. The Most Significant Bit (MSB) of the LFSR is transmitted to the communication channel. [3, 4] This can be implemented using D-Type Flip Flops in Quartus Prime. Typically, a much larger LSFR would be used to generate a PRBS in order to achieve a higher

degree of randomness, as demonstrated in the Serial Communication BER Tester mentioned previously. [2]

3.2.1 Push Button Clock

For testing purposes, the clock will initially be tied to a push button on the DE0-Nano with the status of the LFSR tied to LEDs. Refer to Figure 18 in the appendices for the Quartus Prime Lite logic diagram. The initial status of the LFSR is set to 7 ones. The buttons on the FPGA are active low meaning they become logic level 0 when pressed and logic level one when released. The LSFR status varies when the button is released, not when pressed down. This means the PRBS generator is synchronised to the rising edge of the clock signal which is the expected behaviour. Once the button is pressed 127 times, the LSFR's status returns to the original state of 7 ones. Hence, PRBS generator is producing a repetitive bit sequence of length 127, which is also expected.

When the initial seed is 7 ones, the expected repeating 127 bit sequence is the following.

```
1111111000000100000110000101000111100100010110011101010011111010000111000100
100110110101101111011000110100101110111001100101010
```

This can be compared to the output of the PRBS generator viewed on an oscilloscope.

3.2.2 500kHz Clock

The 50MHz oscillator on the FPGA is initiated and connected to a Phase Locked Loop (PLL) to divide it by 100, producing a 500kHz clock signal. This signal is fed into the clock input of the PRBS generator. Refer to Figure 19 in the appendices for the Quartus Prime Lite logic diagram.

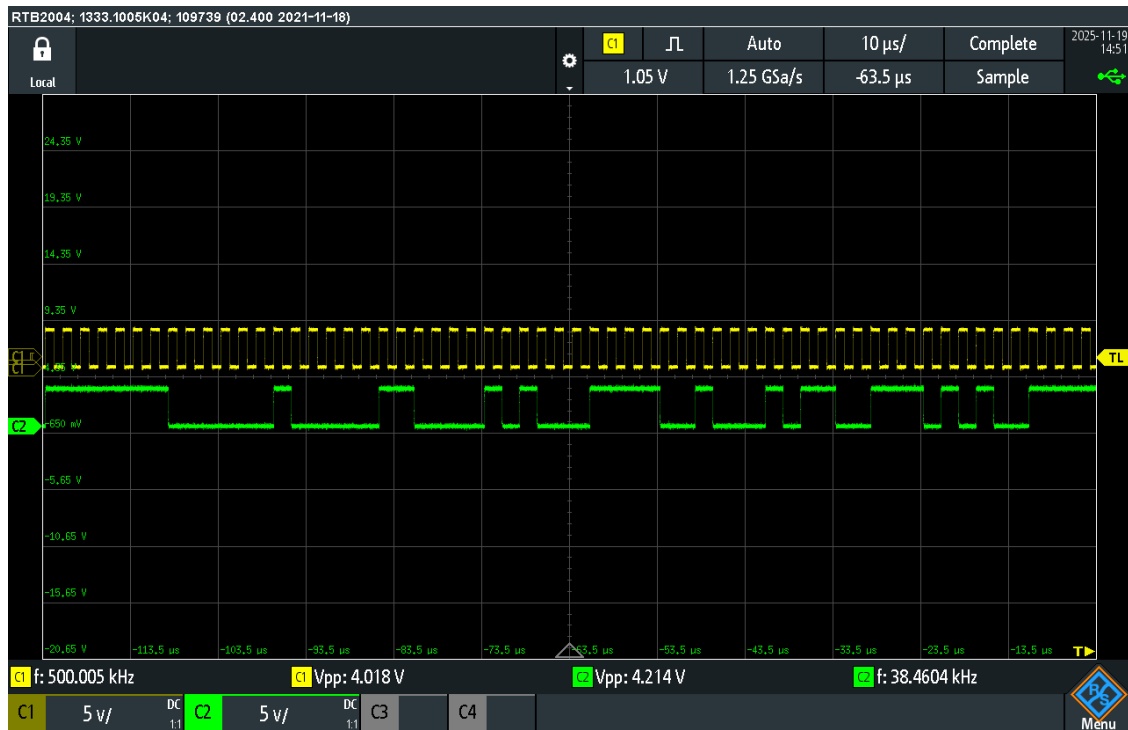


Figure 8: Oscilloscope output of PRBS generator at 500kHz (first 60 bits)

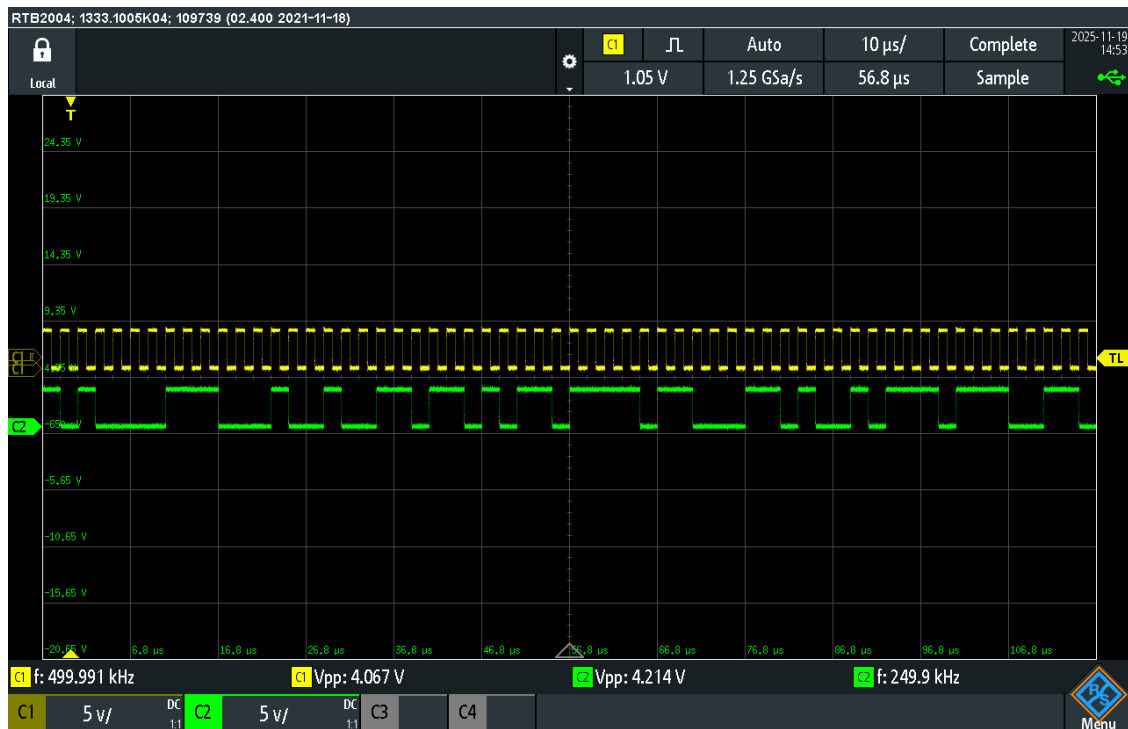


Figure 9: Oscilloscope output of PRBS generator at 500kHz (61st to 120th bits)



Figure 10: Oscilloscope output of PRBS generator at 500kHz. First 7 bits in the figure are the 121st to 127th bits of the sequence. Sequence repeats after this

In Figure 8, Figure 9 and Figure 10 channel 1 (CH1) is tied to the 500kHz clock signal and channel 2 (CH2) is tied to the output of the PRBS generator. At 500kHz, the oscilloscope probes don't distort the square waveforms. When comparing the output to the expected bit sequence, no mistakes are found and the sequence repeats every 127 bits as expected.

3.2.3 10MHz Clock

The 50MHz oscillator on the FPGA is initiated and connected to a PLL to divide it by 5, producing a 10MHz clock signal. This signal is fed into the clock input of the PRBS generator. Refer to Figure 20 in the appendices for the Quartus Prime Lite logic diagram.

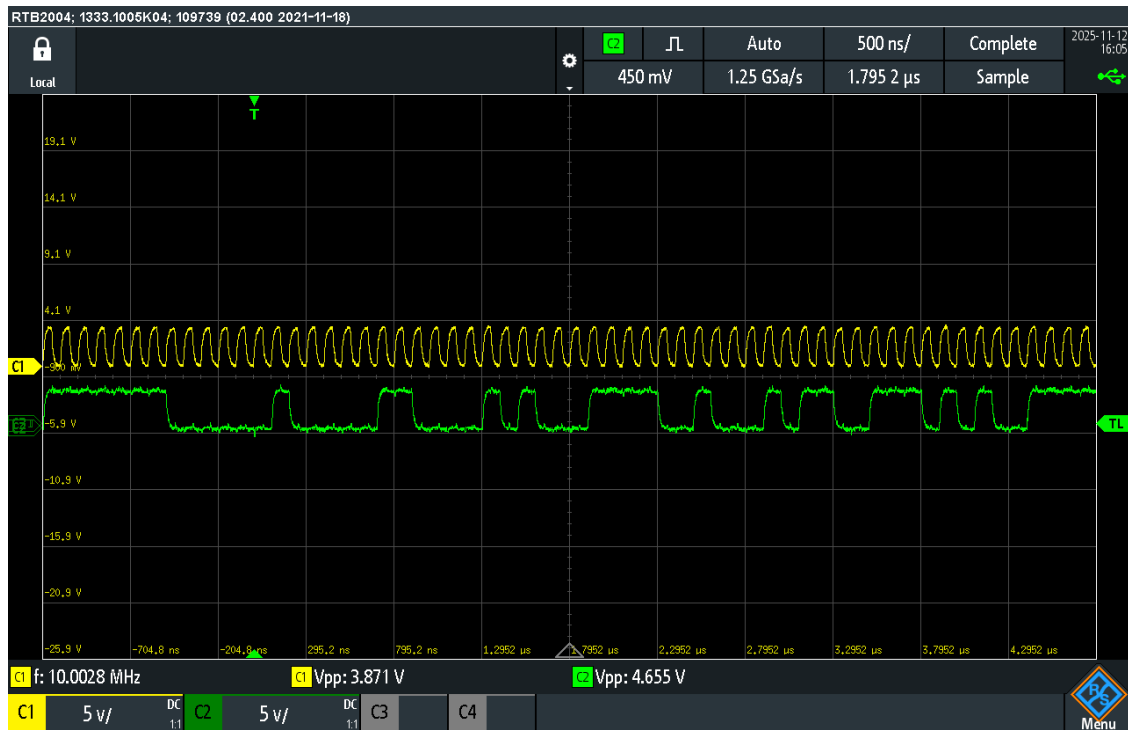


Figure 11: Oscilloscope output of PRBS generator at 10MHz (first 60 bits)

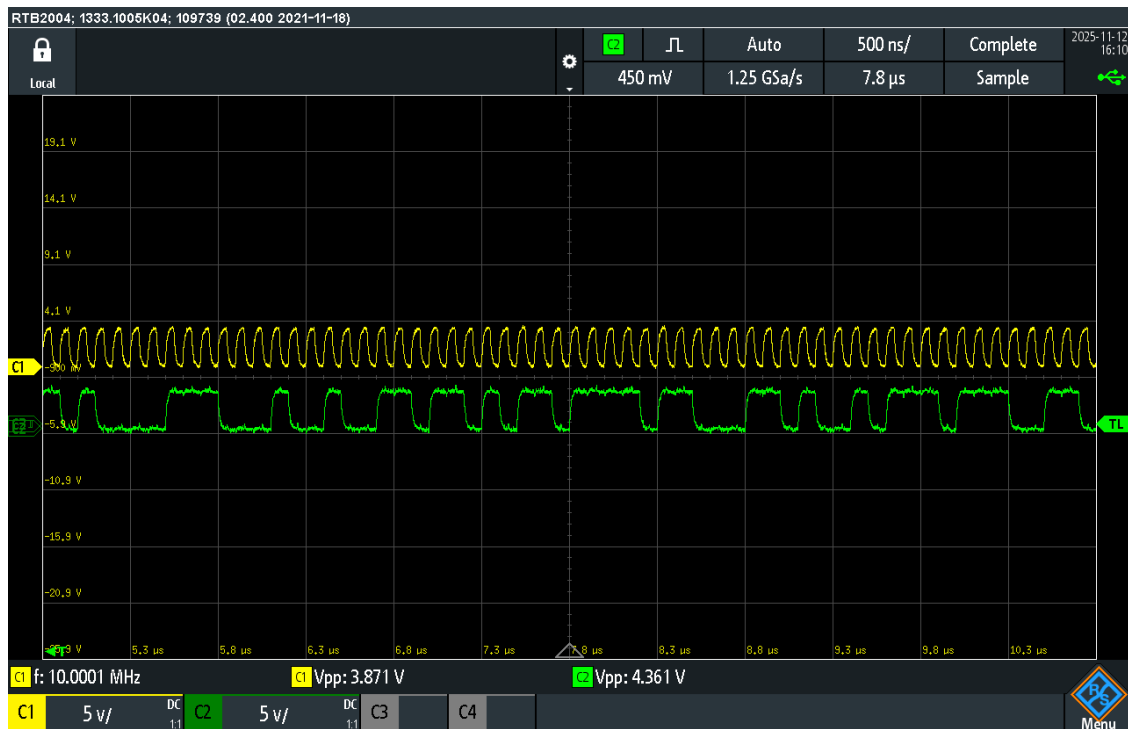


Figure 12: Oscilloscope output of PRBS generator at 10MHz (61st to 120th bits)

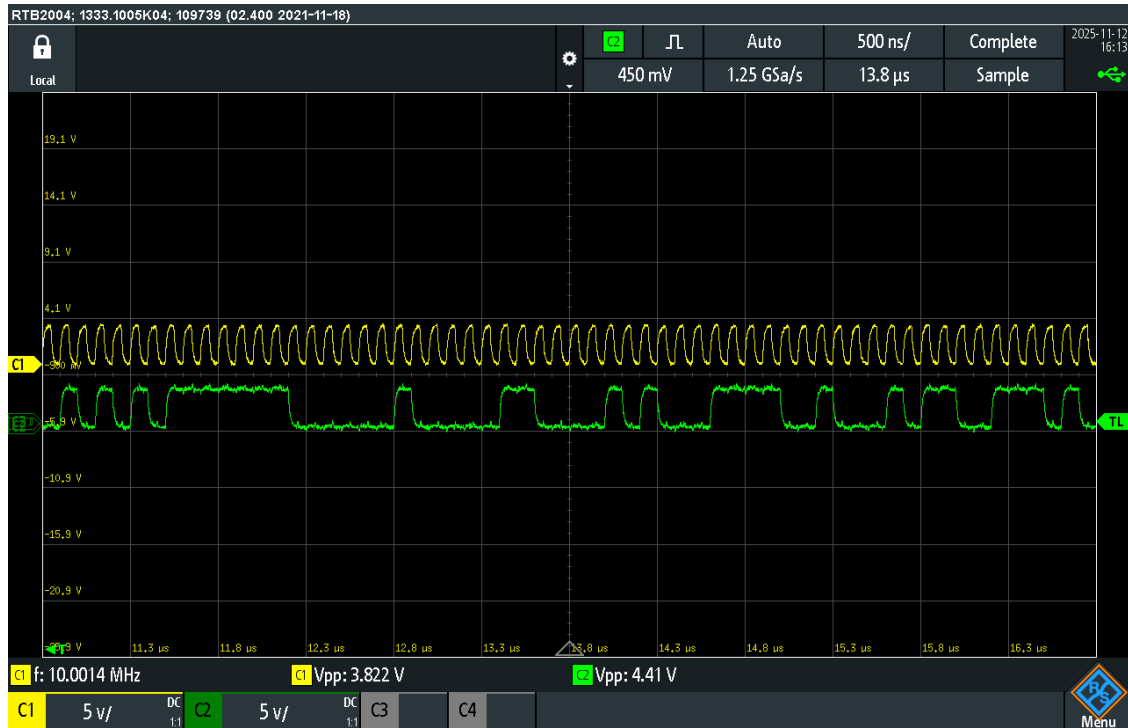


Figure 13: Oscilloscope output of PRBS generator at 10MHz. First 7 bits in the figure are the 121st to 127th bits of the sequence. Sequence repeats after this

In Figure 11, Figure 12 and Figure 13, CH1 is tied to the 10MHz clock signal and CH2 is tied to the output of the PRBS generator. At 10MHz, the oscilloscope probes distort the square waveforms due to high frequency components of the square wave being attenuated. The probe used has a -3dB bandwidth of 10MHz. [9, 10] No mistakes are found when comparing the output to the expected bit sequence and the sequence repeats every 127 bits as expected.

3.2.4 20MHz Clock

The PLL to multiply the 50MHz oscillator by $\frac{2}{5}$, producing a 20MHz clock signal. This signal is fed into the clock input of the PRBS generator. Refer to Figure 21 in the appendices for the Quartus Prime Lite logic diagram.

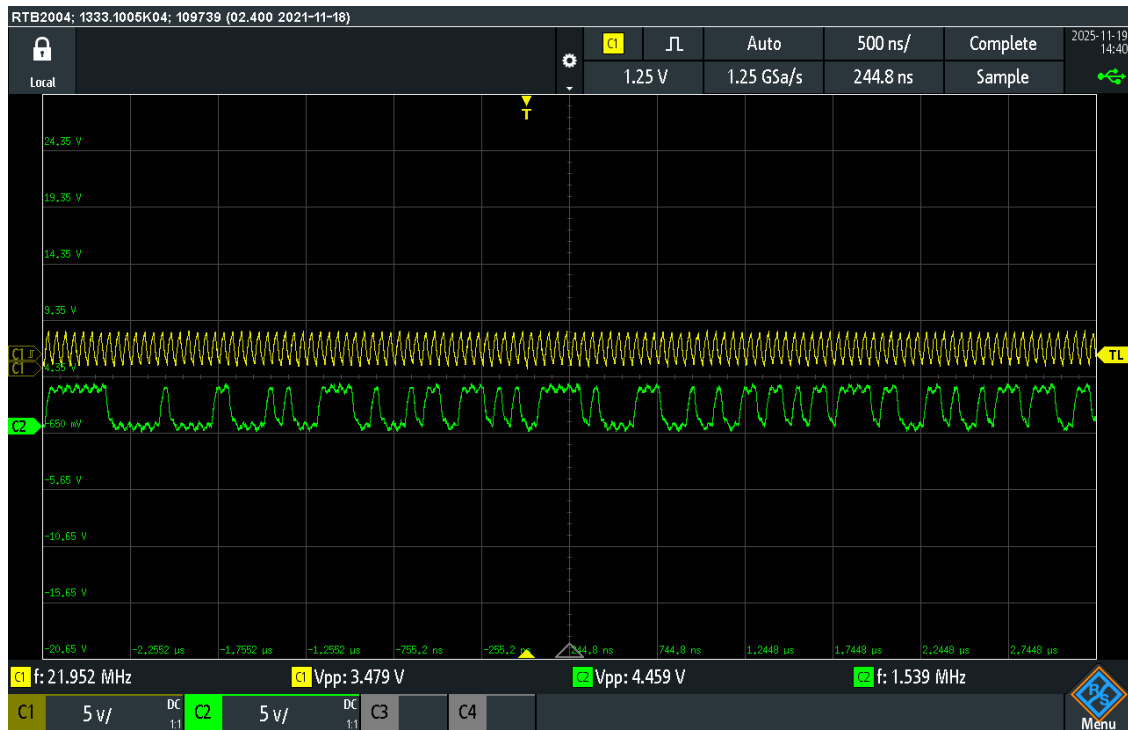


Figure 14: Oscilloscope output of PRBS generator at 20MHz (first 120 bits)

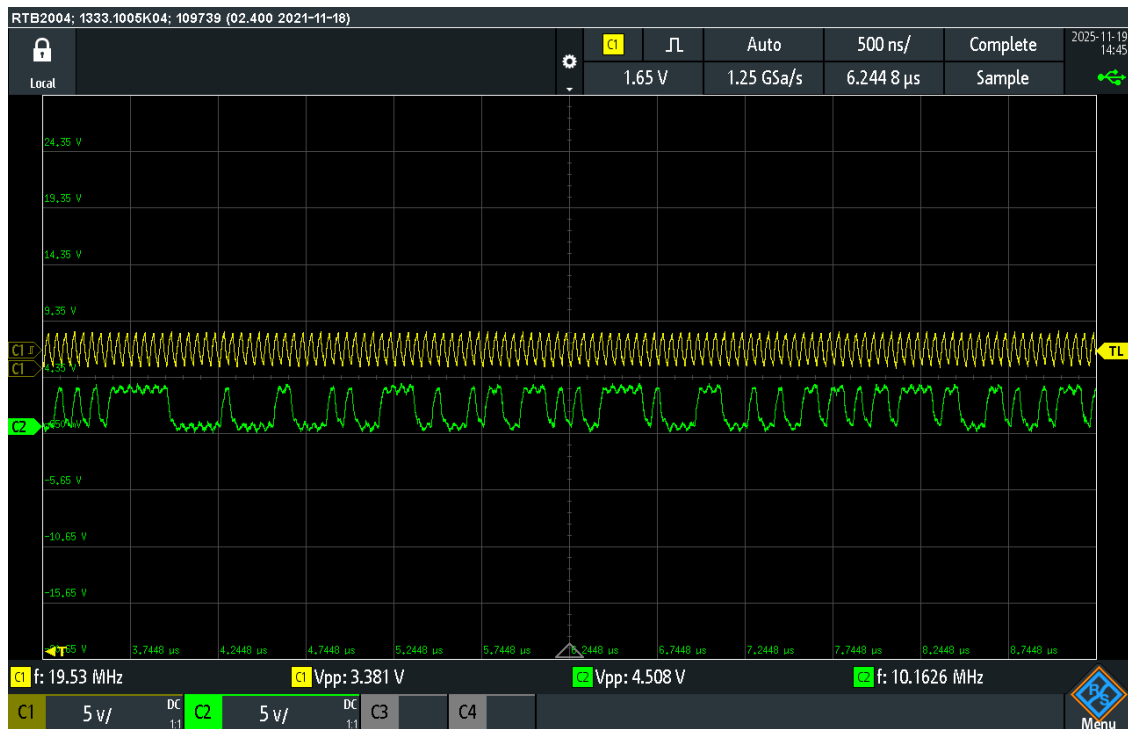


Figure 15: Oscilloscope output of PRBS generator at 20MHz First 7 bits in the figure are the 121st to 127th bits of the sequence. Sequence repeats after this

In Figure 14 and Figure 15, CH1 is tied to the 20MHz clock signal and CH2 is tied to the output of the PRBS generator. At 20MHz, the oscilloscope probes distort the square waveforms due to high frequency components of the square wave being attenuated. This happens at a much greater degree than at 10MHz. [10] The bit sequence produced at 20MHz is the same as the expected bit sequence.

4 Project Management

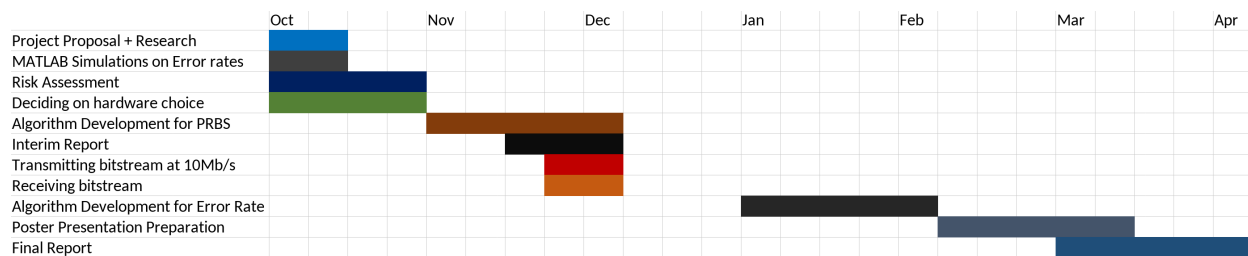


Figure 16: Original Gantt Chart for objectives in Terms 1 and 2

The project has mostly gone to plan with tasks like the risk assessment and hardware choice being completed slightly quicker than expected.

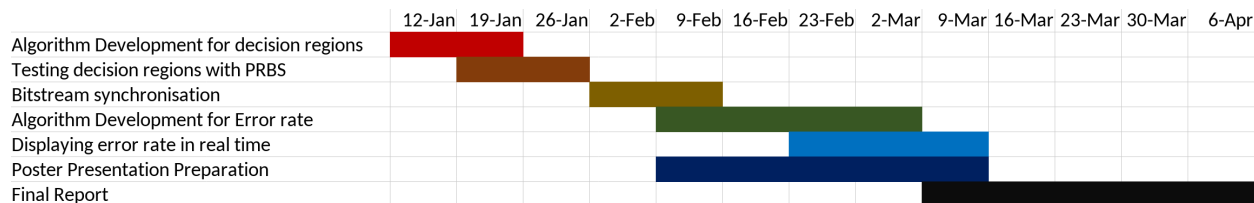


Figure 17: Modified Gantt Chart for objectives in Term 2

The tasks for Term 2 have been broken down further. The most time has been allocated to developing an algorithm for the error ratio calculation since it is the primary goal of this project. Algorithm development for decision regions is not a priority and will be disregarded if reasonable progress isn't made.

5 Future Work

References

- [1] S. M. J. Alam, M. R. Alam, G. Hu, and M. Z. Mehrab, "Bit Error Rate Optimization in Fiber Optic Communications," *International Journal of Machine Learning and Com-*

- puting, vol. 1, no. 5, pp. 435–440, Dec. 2011, doi:<https://doi.org/10.7763/IJMLC.2011.V1.65>.
- [2] M. J. Basford, A. E. Pena-Quintal, S. Greedy, M. Sumner, and D. W. P. Thomas, “An Open Source, FPGA-Based Bit Error Ratio Tester for Serial Communications,” in *2020 International Symposium on Electromagnetic Compatibility - EMC EUROPE*, Rome, Italy, Sep. 2020, pp. 1–6, doi:<https://doi.org/10.1109/EMCEUROPE48519.2020.9245786>.
 - [3] D. Riccardi and P. Novellini, “An Attribute-Programmable PRBS Generator and Checker,” AMD, Santa Clara, California, Tech. Rep. v1, 2011, Available: https://docs.amd.com/v/u/en-US/xapp884_PRBS_GeneratorChecker, (Accessed Dec. 05 2025).
 - [4] M. George and P. Alfke, “Linear Feedback Shift Registers in Virtex Devices,” AMD, Santa Clara, California, Tech. Rep. v1.3, 2007, Available: <https://docs.amd.com/v/u/en-US/xapp210>, (Accessed Dec. 05 2025).
 - [5] S. Chatterjee and B. Tiru, “Design and testing of a bit error rate tester with application to a visible light communication system,” *Materials Today: Proceedings*, (In Press), Apr. 2023, doi:<https://doi.org/10.1016/j.matpr.2023.03.786>.
 - [6] N. Marda, G. Vaishnav, and U. S. N. Rao, “Bit Error Rate Testing Scheme for Digital Communication Devices,” in *The 2014 International Conference on Control, Instrumentation, Energy and Communication (CIEC)*, Calcutta, India, Jan. 2014, pp. 490–493, doi:<https://doi.org/10.1109/CIEC.2014.6959137>.
 - [7] STMicroelectronics, “NUCLEO-xxxxCx, NUCLEO-xxxxRx, NUCLEO-xxxxRx-P, NUCLEO-xxxxRx-Q Data brief,” Datasheet for NUCLEO-F401RE board, Feb. 2014, [Revised Jan. 2025]. Available: https://www.st.com/resource/en/data_brief/nucleo-f401re.pdf, (Accessed Dec. 05 2025).
 - [8] —, “STM32F401xD STM32F401xE,” Datasheet for STM32F401RE Microcontroller, Jan. 2014, [Revised Feb. 2014] Available: <https://www.st.com/resource/en/datasheet/stm32f401re.pdf>, (Accessed Dec. 05 2025).
 - [9] Rohde and Schwartz, “R&S RT-ZP03 Manual,” User Manual for RT-ZP03 Oscilloscope Probe, May 2019, [Revised May 2019] Available: https://www.rohde-schwarz.com/ae/manual/r-s-rt-zp03-manual-manuals_78701-172932.html, (Accessed Dec. 05 2025).
 - [10] J. A. Svoboda and R. C. Dorf, “Fourier Series and Fourier Transform,” in *Introduction to Electric Circuits*, 9th ed., Daniel Sayre and Wendy Ashenberg, Ed. New Jersey: Hoboken: John Wiley & Sons, 2013, pp. 741–802.
 - [11] TerasicTechnologies, “DE0-Nano User Manual,” Datasheet for DE0-Nano EP4CE22F17C6, Jan. 2011, [Revised Mar. 2021] Available: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=139&No=593&PartNo=4#contents>, (Accessed Dec. 05 2025).

A Appendices

A.1 Quartus Prime Lite Block Diagrams

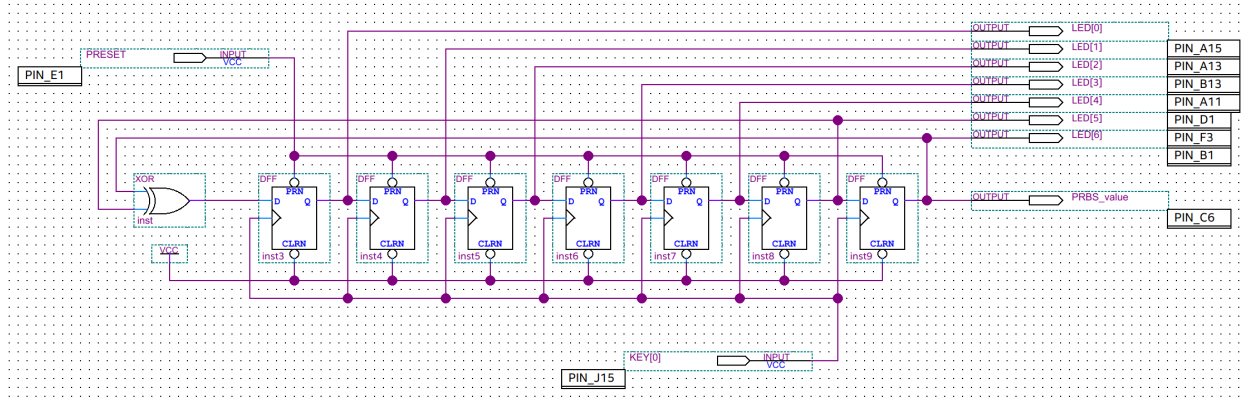


Figure 18: PRBS Generator implemented on Quartus Prime Lite with the clock tied to a push button

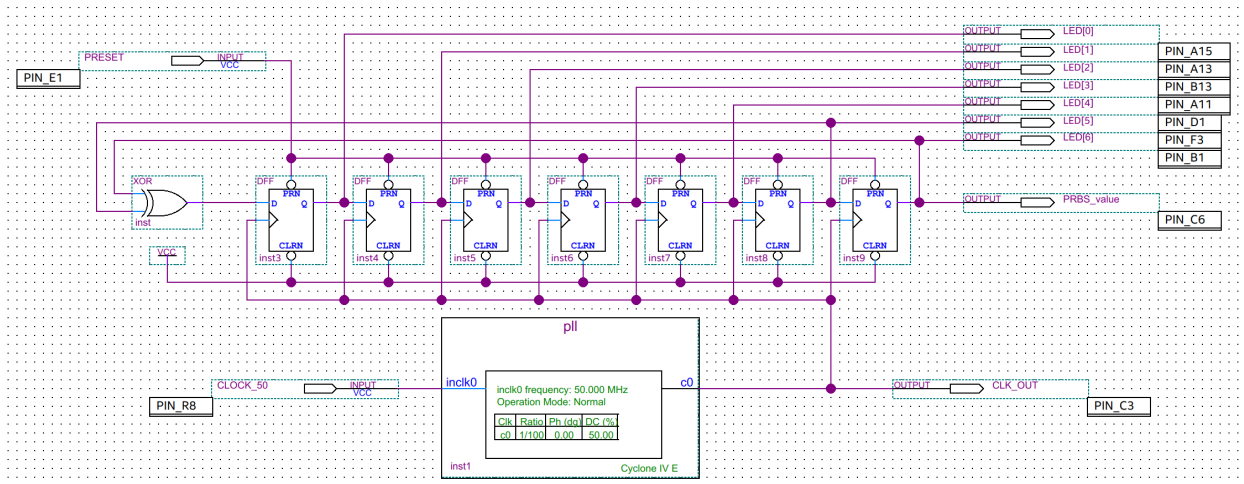


Figure 19: PRBS Generator implemented on Quartus Prime Lite with a 500kHz clock signal

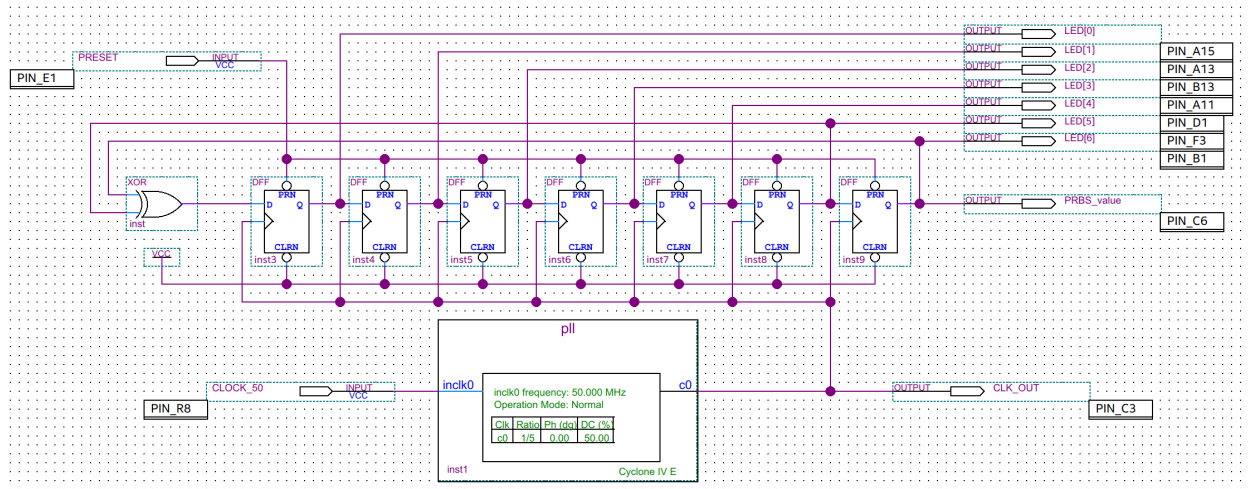


Figure 20: PRBS Generator implemented on Quartus Prime Lite with a 10MHz clock signal

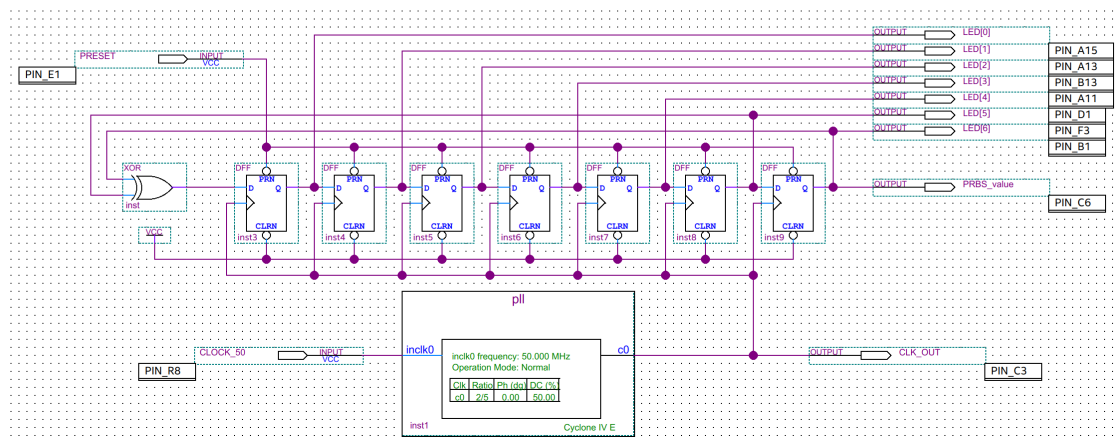


Figure 21: PRBS Generator implemented on Quartus Prime Lite with a 20MHz clock signal